phensim: genotype, phenotype and summary-statistic simulation for statistical genetics

Technical report

phensim 1.0.1: API and numerical contracts

October 2026

Abstract

phensim simulates genotypes with linkage disequilibrium (LD), phenotypes with known genetic architecture, GWAS summary statistics, reference panels, pedigrees and kinship matrices. The core package requires only NumPy; Numba and msprime provide optional acceleration and a second coalescent backend.

The package includes a built-in implementation of Hudson's coalescent with recombination. We compare it with msprime over 30 seeds using site counts, nucleotide diversity and LD decay. A matrix-free draw generates trait backgrounds from the scaled genomic relationship matrix used by the mixed model. The summary-statistic functions also reproduce the seeded draws of the sibling benchmark code they replace.

This report documents the models and numerical conventions and summarizes the validation measurements and their computational cost. The API discussion covers later additions and repairs. Tables and figures retain the archived phensim 1.0.0.dev1 evidence at revision `a7371c5`; those measurements were not rerun for this report revision.

Development note: parts of phensim's code, tests, documentation and this report were produced with AI assistance under human review.

1 Introduction

Simulation with known generating values allows bias, calibration, power and computational cost to be measured before a method is applied to real data. For polygenic scores, fine-mapping, heritability and genetic-correlation estimators, this requires control over LD, allele frequencies and genetic architecture. Summary-statistic methods also require the regression-with-summary-statistics (RSS) model [14] and a way to represent finite reference panels.

phensim began as a teaching snippet and now hosts simulators previously duplicated in the benchmark suites of ldpred3, bipred, gwfm, g2gfun, multipgs and ltpred. The implementation follows the models used by those estimators: trait backgrounds use the scaled GRM, and summary statistics use the RSS likelihood. Returned causal sets, effects, liabilities and relationship matrices make the generating values available to the analysis. The core depends only on NumPy. Seeded draw orders are documented, malformed inputs are rejected, and tiled validation and `int8` genotypes limit temporary storage.

2 Package overview

Table 1 lists the public API. Discrete genotype simulators return sample-major `int8` dosages in $\{0, 1, 2\}$ with columns in physical order, or phased alleles when requested. Gaussian predictors are continuous `float64` draws. LD is passed as a list of dense (R, ix) blocks that tile the variants exactly once. Figure 1 contrasts the LD of the three structured genotype models.

Table 1: Public API by module.

Module	Functions
genotypes	`simulate_independent`, `simulate_population_structure`, `simulate_ar1_blocks`, `realistic_block_sizes`, `simulate_haplotype_blocks`, `simulate_coalescent`, `simulate_by_mutation_rate`
phenotypes	`simulate_trait`, `simulate_binary_trait`, `simulate_confounded_trait`, `simulate_gxe_trait`, `simulate_correlated_traits`, `ascertain_case_control`, `h2_liability`, `n_eff_case_control`
ancestry	`drift_frequencies`, `simulate_populations`, `simulate_admixed`, `simulate_split_coalescent`, `simulate_genome`, `simulate_meta_sumstats`
hapnest	`simulate_hapnest`, `iter_hapnest`
sumstats	`simulate_effects`, `simulate_effects_pair`, `genetic_correlation`, `simulate_sumstats`, `simulate_sumstats_pair`, `shake_ld`, `prepare_blocks`, `simulate_gaussian_individuals`, `gwas_scan`
pedigree	`simulate_pedigree`, `pedigree_birth_times`, `kinship_from_pedigree`, `mendelian_draw`
kinship	`grm`, `ibs_kinship`, `loco_kinships`, `iter_loco_kinships`, `windowed_kinships`
io	`write_plink`

3 Genotype models

Independent and structured SNPs. `simulate_independent` draws $G_{ij} \sim \mathrm{Bin}(2, p_j)$, where p_j is either a fixed minor allele frequency or drawn from a named frequency spectrum (Beta, uniform, rare or common). Each site's minor allele is the counted allele with probability 1/2.

`simulate_population_structure` draws base frequencies

$$p_j = \mathrm{clip}\big(\mathrm{maf} + N(0, 0.05^2),\, 0.05,\, 0.95\big)$$

and then, independently for each of K populations, either the normal approximation $p_{jk} \sim N(p_j, F p_j(1 - p_j))$ or the exact Balding–Nichols distribution [1]

$$p_{jk} \sim \mathrm{Beta}\Big(\tfrac{p_j(1-F)}{F}, \tfrac{(1-p_j)(1-F)}{F}\Big),$$

clipped to $[0.01, 0.99]$.

AR(1) haplotype blocks. Within a block of k SNPs, each individual carries two latent haplotypes $z \sim N(0, C)$ with $C_{ij} = \rho^{|i-j|}$. A haplotype carries the allele at SNP j when $z_j > \Phi^{-1}(1 - f_j)$, so the allele frequency is exactly f_j; the dosage sums the two haplotypes. The default draws $z = L\varepsilon$ with $LL^\top = C + 10^{-8} I$. This is the family's historical stream, which phensim reproduces bit for bit. The alternative recursion $z_j = \rho z_{j-1} + \sqrt{1 - \rho^2}\, \varepsilon_j$ costs $O(nk)$, needs no factorization, and is exact at $\rho = \pm 1$. Both consume the same normals. `realistic_block_sizes` draws log-normal block lengths with a chosen coefficient of variation, which stresses the few large blocks that dominate quadratic LD work.

Founder haplotype copying. Each block has a small set of Bernoulli(0.3) founder haplotypes. Every individual copies two founders, with per-site flips at a mutation rate. The implementation requires one pass over the founders and output.

Coalescent genotypes. `simulate_coalescent` grows a segment until at least m SNPs pass the MAF filter. It starts at the larger of 1 Mb and $m/1200$ Mb and multiplies the length by 1.8 per retry. It then keeps the first m of those SNPs in physical order and cuts them into contiguous blocks. `simulate_by_mutation_rate` instead fixes the segment and recombination rate, so the genealogy is determined by the seed and the mutation rate alone controls SNP density. The family's scaling benchmarks use this fixed-genealogy design.

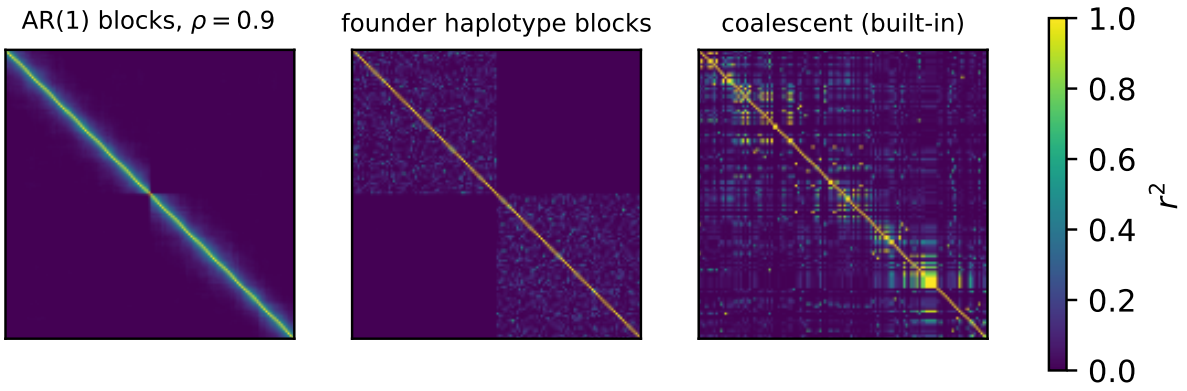


Figure 1: Squared correlation between 120 consecutive SNPs in 1000 simulated individuals. AR(1) blocks have smooth geometric decay and hard block edges. Founder haplotype copying produces patchy haplotype sharing. The coalescent produces recombination-driven, irregular LD whose block boundaries are not imposed.

4 The built-in coalescent engine

The engine is a compact implementation of Hudson's coalescent with recombination [7] for one constant-size diploid population, written to compile end to end under Numba. It writes dosages directly instead of building a tree-sequence table collection.

Ancestry. The simulation starts from $2n$ haploid lineages carrying $[0, L)$. With k lineages, the next event occurs after an exponential waiting time at total rate

$$\lambda = \binom{k}{2} \frac{1}{2N_e} + r \sum_i \ell_i,$$

where ℓ_i is the distance between lineage i's leftmost and rightmost ancestral positions. A recombining lineage is chosen in proportion to ℓ_i through a Fenwick tree [4], in $O(\log k)$. The breakpoint is uniform on that span, gaps included, and is clamped to the representable interior of the span.

Coalescence merges the two segment lists. Each overlapping interval creates a node and two edges, and each segment carries the number of samples below it. An interval whose count reaches $2n$ has found its most recent common ancestor and leaves the simulation, which is what makes the process terminate.

Mutations and genotypes. Each edge receives Poisson($\mu\, t_{\text{branch}}\, \text{span}$) infinite-sites mutations at uniform positions. Densification then sweeps the marginal trees left to right, inserting edges by (left, parent time) and removing them by (right, $-$parent time), as in tskit. Each mutation adds one to every sample below its node, and haplotypes $2i$ and $2i + 1$ form individual i.

Engineering. All work arrays are preallocated from expected event counts. On overflow the engine doubles the buffer and reruns with the same seed, which reproduces the same events, so a caller never sees a partial result. The ancestry seed is the caller's seed masked to 31 bits, and the mutation stream uses $(s \cdot 2654435761) \bmod 2^{31}$. The public functions accept seeds in $[1, 2^{31})$, the range both backends honour. Without Numba the same code runs as pure Python on NumPy's global generator, whose state is saved and restored around each call. Linked-list and Fenwick-weight invariants are asserted at every event boundary in the test suite.

Agreement with msprime. msprime [9, 2] is the reference implementation; phensim calls it on a continuous genome with the binary mutation model. We simulated $n = 100$ diploids on 200 kb with $N_e = 10^4$ and $r = \mu = 10^{-8}$, for 30 seeds per backend.

Table 2 shows that site counts and nucleotide diversity agree with each other and with the neutral expectations $\mathbb{E}[S] = \theta \sum_{i<2n} 1/i$ and $\mathbb{E}[\pi] = \theta$, where $\theta = 4N_e\mu L$. Figure 2 shows the decay of r^2 with physical distance: in no distance bin do the backends differ by more than 1.6 standard errors. These

comparisons cover selected summaries, not all properties of the sampler. The backends draw different events from the same model.

Table 2: Coalescent backends, mean over 30 seeds (standard error).

Backend	Segregating sites (SE)	Diversity $\sum 2f(1-f)$ (SE)
built-in (Numba)	478 (9)	82 (2)
msprime 1.4	477 (6)	82 (2)
neutral expectation	470	80

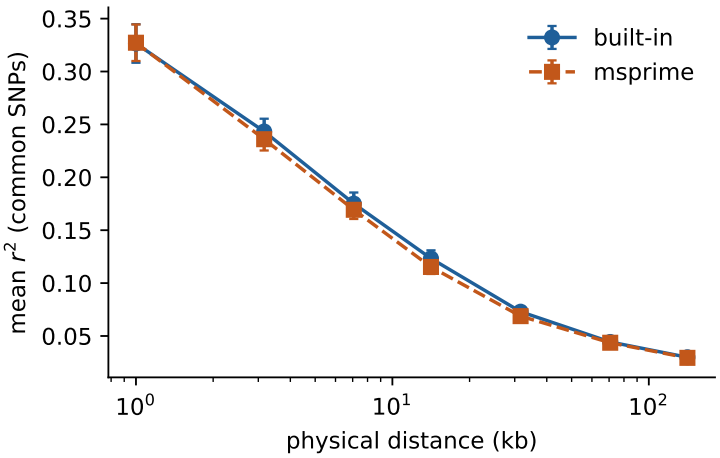


Figure 2: Mean r^2 between common SNPs (MAF ≥ 0.05) by physical distance, for the built-in engine and msprime with the same design as Table 2. Bars are 95% intervals over seeds.

5 Kinship

`grm` standardizes each variant over its called samples (missing calls contribute zero) [12] and forms $ZZ^\top/m$. It then applies the EMMAX scaling [8]: subtract the mean off-diagonal element, then divide by the mean diagonal. Columns with no variation among called dosages contribute zero, including fractional constants for which centering alone would leave roundoff.

Leave-one-chromosome-out matrices subtract each chromosome's Gram from the full Gram under the same global standardization, so they are exact rather than re-estimated. `windowed_kinships` splits the Gram the same way into a local window and its complement. `ibs_kinship` is the per-pair fraction of identical called genotypes, computed with one-hot GEMMs.

6 Phenotype models

`simulate_trait` builds $\ell = u + q + e$. The infinitesimal background is $u \sim N(0, \sigma_u^2 K)$, where K is the scaled GRM a mixed model would fit. The QTL component q comes from causal variants, and the residual is $e \sim N(0, 1 - h^2)$. Three architectures split h^2: `mixed` gives half to u and half to q, `infinitesimal` gives all of it to u, and `qtl` gives all of it to q. QTL effects are normal or of equal magnitude, act on standardized causal columns, and are rescaled so that the in-sample $\mathrm{Var}(q)$ equals its target exactly.

Proposition 1 (Matrix-free background)**.** *Let Z be the called-standardized genotypes, whose columns sum to zero, and let $S = \sum_{ij} Z_{ij}^2$. The EMMAX-scaled GRM equals $K = \frac{n-1}{S} ZZ^\top + \frac{1}{n} J$. Hence*

$$u = \sigma_u \left(\sqrt{\tfrac{n-1}{S}}\, Zw + \tfrac{c}{\sqrt{n}} \right), \qquad w \sim N(0, I_m),\ c \sim N(0,1),$$

has covariance exactly $\sigma_u^2 K$.

Proof. Write $K_0 = ZZ^\top/m$. Its grand sum is $\mathbf{1}^\top ZZ^\top \mathbf{1}/m = 0$ and its trace is S/m. The mean off-diagonal element is therefore $-S/(mn(n-1))$. After subtracting it, the mean diagonal is $S/(m(n-1))$, and dividing by it gives the stated K. The factor $F = [\sqrt{(n-1)/S}\, Z,\ \mathbf{1}/\sqrt{n}]$ satisfies $FF^\top = K$. □

The draw therefore needs $m+1$ normals and one matrix–vector product, with no $n \times n$ matrix and no eigendecomposition. Its in-sample variance has expectation $\sigma_u^2(1-1/n)$, because $\mathbf{1}^\top K \mathbf{1} = n$. A supplied K is drawn through its eigendecomposition instead, and the two give the same distribution: in Table 3 the background variances are 0.504 and 0.512 against an expectation of 0.500.

Derived traits.

- **Binary traits** threshold the liability at $\Phi^{-1}(1-K)$.
- **Confounded traits** add a share s of variance along the leading eigenvector of K, used here as the structure axis, and reuse its eigendecomposition for the background.
- **Gene–environment traits** add standardized causal genotypes multiplied by a standardized environment, scaled to the interaction variance.
- **Correlated traits** share both components across traits: $g_b \propto r_g u_1 + \sqrt{1-r_g^2}\, u_2 + r_g q_1 + \sqrt{1-r_g^2}\, q_2$. This targets the *total* genetic correlation and is exact at $r_g = \pm 1$.

`ascertain_case_control` samples exact case and control counts, and `h2_liability` implements the observed-to-liability transformation of Lee et al. [10]. It warns unless the study case fraction is given.

Table 3 compares realized and target quantities over 40 draws on a coalescent panel of 1500 individuals and 3000 SNPs. Every target is recovered within simulation error. The realized variance shares are fractions of the realized liability variance, so they fluctuate with the sample covariances between components.

Table 3: Phenotype simulators: target versus realized. For `simulate_trait` the three numbers are the background, QTL and total genetic shares of liability variance.

Simulator and quantity	Target	Realized mean (SD), 40 draws
`simulate_trait` (mixed)	0.25 / 0.25 / 0.50	0.25 (0.03) / 0.25 (0.02) / 0.50 (0.03)
`simulate_trait` (infinitesimal)	0.50 / 0.00 / 0.50	0.50 (0.04) / 0.00 (0.00) / 0.50 (0.04)
`simulate_trait` (qtl)	0.00 / 0.50 / 0.50	0.00 (0.00) / 0.51 (0.01) / 0.51 (0.01)
background variance, K omitted / supplied	0.500	0.504 (0.074) / 0.512 (0.081)
`simulate_binary_trait` prevalence	0.05	0.049 (0.007)
`simulate_binary_trait` prevalence	0.20	0.201 (0.012)
`simulate_correlated_traits` r_g	0.00	-0.023 (0.136)
`simulate_correlated_traits` r_g	0.50	0.476 (0.095)
`simulate_correlated_traits` r_g	0.90	0.896 (0.016)
`simulate_gxe_trait` interaction share	0.20	0.199 (0.012)

7 Summary statistics

The RSS model. With standardized effects β and block-diagonal population LD R, marginal GWAS effects are drawn as [14]

$$\hat{\beta} = R\beta + \varepsilon, \qquad \varepsilon \sim N(0, R/n),$$

or $N(0, DRD)$ with $D_{jj} = n_j^{-1/2}$ for per-variant sample sizes. Each block contributes $\varepsilon = Fz/\sqrt{n}$ with $z \sim N(0, I_r)$. The factor F is chosen per call:

- by default, the exact factor of R: Cholesky, or $V\Lambda^{1/2}$ for singular LD, with no diagonal noise added;
- with `jitter`, the Cholesky factor of $R + \epsilon I$;
- with `factors`, matrices supplied by the caller, for example an eigenvalue-clipped root of thresholded LD, which need not be PSD.

The signal always uses R. `prepare_blocks` validates and factors the blocks once into a read-only snapshot that every consumer accepts.

Effects. `simulate_effects` draws a sparse, polygenic, equal-magnitude or MAF-dependent shape and rescales it so that $\beta^\top R\beta = h^2$ exactly. The MAF-dependent shape uses $\beta_j \propto [2f_j(1-f_j)]^{(1+\alpha)/2}$, so the per-allele variance is proportional to $[2f_j(1-f_j)]^\alpha$. This is the convention of LDAK [11], SBayesS [13] and ldpred3, in which $\alpha = -1$ means no MAF dependence.

`simulate_effects_pair` draws shared effects with correlation ρ and scales each trait to its own h^2. The causal layout is either a shared Bernoulli(p) set, or exact per-trait and shared counts (n_a, n_b, n_{ab}); the latter is the bivariate causal mixture of MiXeR [5], whose target genetic correlation is $\rho\, n_{ab}/\sqrt{n_a n_b}$. `genetic_correlation` returns the correlation realized under the LD.

Two studies and reference panels. `simulate_sumstats_pair` draws z_a and then z_b per block and sets $\varepsilon_b = F(\rho z_a + \sqrt{1-\rho^2} z_b)/\sqrt{n_b}$, which gives cross-covariance $\rho R/\sqrt{n_a n_b}$. Here ρ is the sampling-noise correlation, not the fraction of overlapping participants.

`shake_ld` simulates a reference panel: it draws $X = ZF^\top$ with n_{ref} rows, standardizes the columns, and returns $(1-s)X^\top X/n_{\text{ref}} + sI$. An optional chunked path accumulates the same draws with the stable one-pass update of Chan et al. [3], bounding memory at $O(\text{chunk} \cdot k + k^2)$.

Individual-level GWAS. `gwas_scan` fits each variant on its called samples. It reports the Pearson correlation r, the standard error $\sqrt{(1-r^2)/(n-2)}$, the OLS t statistic and a normal p-value. A variant whose called genotypes or called phenotypes are constant, tested exactly, returns NaN instead of a roundoff statistic.

Validation tolerances. Blocks must tile the variants and be finite, symmetric, unit-diagonal, bounded in $[-1, 1]$ and PSD. Validation runs in tiles of 256 rows, so genome-wide LD is never assembled. Tolerances follow the input precision: the relative tolerance is $\max(10^{-7}, 4\epsilon)$, and negative eigenvalues are clipped up to $k \max(64\,\epsilon_{64} \max(1, \lambda_{\max}), \epsilon_{\text{in}})$. float32 LD, the family's usual storage, is therefore judged at float32 roundoff.

Table 4 confirms each property against its theoretical expectation. The empirical noise covariance matches R/n to the error expected of a Wishart estimate. Panel noise matches the large-sample standard deviation of a sample correlation, $(1-\rho^2)/\sqrt{n_{\text{ref}}}$. Null scans are calibrated: mean $\lambda_{\text{GC}} = 1.002$ (SD 0.061) over 40 null traits with 5% missing calls.

Table 4: Summary-statistic layer: properties versus expectation.

Check	Expected	Realized
`simulate_effects`: $\lvert\beta^\top R\beta - h^2\rvert$, 4 architectures	0	1.1e-16
`simulate_sumstats`: $\lVert n\,\widehat{\text{Cov}} - R\rVert_F / \lVert R\rVert_F$, 4000 draws	≈ 0.096	0.095
`simulate_sumstats_pair`: noise correlation	0.40	0.399
`simulate_effects_pair`: mean realized r_g (40/40 causal, 20 shared, $\rho = 0.8$)	0.40	0.395 (0.160)
`shake_ld`: off-diagonal RMSE, $n_{\text{ref}} = 100$	≈ 0.096	0.097
`shake_ld`: off-diagonal RMSE, $n_{\text{ref}} = 500$	≈ 0.043	0.045
`shake_ld`: off-diagonal RMSE, $n_{\text{ref}} = 2500$	≈ 0.019	0.020
`gwas_scan`: λ_{GC}, 40 null traits, 5% missing calls	1	1.002 (0.061)

Parity with the family. With `jitter=1e-4`, per-trait sample sizes and `shrink`, these functions reproduce byte for byte:

- ldpred3: `_metrics.sumstats` and `_realistic_ld.panel_genome`;
- bipred: `rg_architectures.sim_effects`, `sumstats_pair`, `ref_panel` and `realized_rg`, and `mixer_overlap._sim_mixture`.

This was checked against the siblings' own source and is pinned by verbatim oracles in `tests/test_family_parity.py`. A sibling can therefore replace its local copy without changing any seeded result.

8 Pedigrees

`simulate_pedigree` produces multi-generation trio tables from founder couples. Each couple has two to four children, and remarriage produces half-siblings. `kinship_from_pedigree` orders individuals parents first and fills the additive relationship matrix with Henderson's tabular recursion [6]:

$$A_{kj} = \tfrac{1}{2}(A_{s_k j} + A_{d_k j}), \qquad A_{kk} = 1 + \tfrac{1}{2} A_{s_k d_k},$$

where unknown parents contribute zero. The cost is $O(n^2)$, and the result is exactly symmetric and includes inbreeding.

`mendelian_draw` samples $a \sim N(0, A)$ in $O(n)$ storage: $a_i = \sum_p a_p/2 + \sqrt{w_i} z_i$, summing over known parents, with $w_i = 1 - \sum_p A_{pp}/4$. This is the Mendelian sampling variance $(1 - (F_s + F_d)/2)/2$ when both parents are known. The required diagonals come from a memoized pair recursion.

Over 2000 draws on a simulated 367-person pedigree, which includes inbred individuals, the empirical covariance matched A with a standardized root-mean-square error of 0.998; pure sampling noise gives 1. `pedigree_birth_times` assigns generation-coherent birth years. It solves relative parent–child constraints in both directions, allowing unrelated founders to enter later while rejecting incompatible generation assignments.

9 Output

`write_plink` writes SNP-major PLINK 1 binary files. Genotypes are coded 00, 10 and 11 for zero, one and two copies of BIM allele 2, with 01 for missing, packed four samples per byte. Every genotype and metadata check, including unique and printable sample identifiers, PLINK chromosome codes and integral positions, completes before any file is opened.

10 Performance

Table 5 gives indicative single-thread costs measured on a shared machine (Darwin arm64, load average 11.5). Both absolute timings and relative costs may differ on other systems.

- At this size the msprime backend is somewhat faster than the built-in engine. The built-in engine's value is that it has no compiled dependency and writes genotypes directly.
- The matrix-free trait draw is about 25 times faster than drawing through a supplied K. The gap widens with n, because the eigendecomposition costs $O(n^3)$ while the matrix-free draw costs $O(nm)$.
- Prepared LD blocks remove the per-call validation and factorization, which dominate repeated summary-statistic draws.
- The AR(1) recursion is not faster at a block size of 200; it matters for very large blocks and exact $\rho = \pm 1$.

Table 5: Indicative wall-clock cost, median of three runs after a warm-up, BLAS pinned to one thread.

Call	Seconds (median of 3)
simulate_coalescent, $n = 1000$, $m = 10^4$, built-in	0.73
simulate_coalescent, $n = 1000$, $m = 10^4$, msprime	0.47
simulate_ar1_blocks, $n = 5000$, $m = 2 \times 10^4$, Cholesky	2.51
simulate_ar1_blocks, $n = 5000$, $m = 2 \times 10^4$, scan	3.51
simulate_trait, $n = 3000$, $m = 5000$, K omitted (matrix-free)	0.22
simulate_trait, $n = 3000$, $m = 5000$, K supplied (eigh)	5.37
grm, $n = 3000$, $m = 5000$	0.43
simulate_sumstats, $m = 5 \times 10^4$ (250 blocks of 200), raw blocks	0.15
simulate_sumstats, same, prepared blocks	0.01
shake_ld, same, $n_{\text{ref}} = 500$	0.29

11 Reproducibility and integration

Seeds. Seeds are integers, `None`, or NumPy `Generators`. A `Generator` continues its stream, so composed simulations remain reproducible.

Population, genome and reference-based models. `drift_frequencies` draws per-population Balding–Nichols or normal drift. `simulate_populations` draws population-specific AR(1) blocks. `simulate_admixed` joins ancestry tracts along a genetic map and returns per-haplotype ancestry. `simulate_split_coalescent` uses msprime for a simultaneous split and an optional admixture pulse. `simulate_genome` seeds independent coalescent chromosomes. `simulate_meta_sumstats` combines independent population GWAS by inverse-variance weighting. See `docs/technical.md` for their models.

`simulate_hapnest` copies phased references using Gamma segment ages and exponential lengths in cM, retaining only alleles older than the segment. It supports discrete donor populations, optional Numba, sample batches and disk-backed output; `iter_hapnest` yields batches directly. Its model and conventions are detailed in `docs/hapnest.md`. `simulate_gaussian_individuals` draws continuous predictors with population covariance R, including singular support, without sample rescaling.

Draw orders. Draw orders are part of the contract, and every change to a seeded output is recorded in the changelog. Results should carry the package version and git revision. The measured evidence in this report is phensim 1.0.0.dev1 at revision `a7371c5`, with NumPy 2.4.6, Numba 0.66.0, msprime 1.4.2 and Python 3.14.6.

Sibling caches. Sibling repositories bind their caches to phensim's source:

- bipred tags cached coalescent segments with a hash of the coalescent, genotype and JIT modules, plus a hand-bumped msprime tag;
- gwfm fingerprints the same files in its simulation cache;
- ldpred3 records a hash of every phensim module in its run archives.

New features that do not touch genotypes therefore live outside those modules. As a further consequence, a change to the coalescent or to the msprime call invalidates downstream caches. Only some of these caches are keyed automatically: in ldpred3, a cache version must be bumped by hand.

12 Limitations

- The built-in coalescent models one constant-size population. The msprime split API adds a simultaneous split and an optional admixture pulse; use msprime directly for other demography, migration or selection.
- Block models have no LD across block boundaries.
- The prevalence target is exact only for Gaussian liabilities.
- The RSS oracle assumes a single homogeneous sample.
- Age-of-onset, follow-up and censoring models remain in ltpred and aadgen. The exact LD of the AR(1) threshold model is not implemented here.

Reproducing this report

```
OPENBLAS_NUM_THREADS=1 OMP_NUM_THREADS=1 python report/make_evidence.py
cd report && tectonic phensim_report.tex
```

The evidence script is fully seeded. It regenerates every table, figure and quoted measurement, and took 116 s on the recording machine for the saved snapshot. Running it now replaces that snapshot with measurements of the current checkout; compiling the TeX alone preserves the saved evidence.

References

[1] D. J. Balding and R. A. Nichols. A method for quantifying differentiation between populations at multi-allelic loci and its implications for investigating identity and paternity. *Genetica* 96:3–12, 1995.

[2] F. Baumdicker et al. Efficient ancestry and mutation simulation with msprime 1.0. *Genetics* 220:iyab229, 2022.

[3] T. F. Chan, G. H. Golub and R. J. LeVeque. Algorithms for computing the sample variance: analysis and recommendations. *The American Statistician* 37:242–247, 1983.

[4] P. M. Fenwick. A new data structure for cumulative frequency tables. *Software: Practice and Experience* 24:327–336, 1994.

[5] O. Frei et al. Bivariate causal mixture model quantifies polygenic overlap between complex traits beyond genetic correlation. *Nature Communications* 10:2417, 2019.

[6] C. R. Henderson. A simple method for computing the inverse of a numerator relationship matrix used in prediction of breeding values. *Biometrics* 32:69–83, 1976.

[7] R. R. Hudson. Properties of a neutral allele model with intragenic recombination. *Theoretical Population Biology* 23:183–201, 1983.

[8] H. M. Kang et al. Variance component model to account for sample structure in genome-wide association studies. *Nature Genetics* 42:348–354, 2010.

[9] J. Kelleher, A. M. Etheridge and G. McVean. Efficient coalescent simulation and genealogical analysis for large sample sizes. *PLoS Computational Biology* 12:e1004842, 2016.

[10] S. H. Lee, N. R. Wray, M. E. Goddard and P. M. Visscher. Estimating missing heritability for disease from genome-wide association studies. *American Journal of Human Genetics* 88:294–305, 2011.

[11] D. Speed, G. Hemani, M. R. Johnson and D. J. Balding. Improved heritability estimation from genome-wide SNPs. *American Journal of Human Genetics* 91:1011–1021, 2012.

[12] J. Yang et al. Common SNPs explain a large proportion of the heritability for human height. *Nature Genetics* 42:565–569, 2010.

[13] J. Zeng et al. Signatures of negative selection in the genetic architecture of human complex traits. *Nature Genetics* 50:746–753, 2018.

[14] X. Zhu and M. Stephens. Bayesian large-scale multiple regression with summary statistics from genome-wide association studies. *Annals of Applied Statistics* 11:1561–1592, 2017.